HXC-108_cli_recordings_evidence

HXC-108 — HelixCode CLI Client: Real Video-QA Recordings (curated evidence)

Run-id	HXC-108_cli_recordings
Date (UTC)	2026-06-22 / 2026-06-23
HEAD	a92a851f03e24080814a2954f7c807b96f8f74d3
Surface	HelixCode CLI client (helix_code/bin/cli)
Capture method	**asciinema** (terminal session → text cast) → agg (GIF) → ffmpeg (MP4 H.264 +faststart yuv420p). **TCC-free**: needs NO macOS Screen-Recording grant (the harness's native screencapture -l<wid> window path is TCC-blocked on this host — see §"Env gap" below).
Recordings dir	/Volumes/T7/Downloads/Recordings/ (gitignored raw corpus, §11.4.128)
Prefix	helixcode (from HELIX_RELEASE_PREFIX in <repo-root>/.env, §11.4.155/.151)
Validator	harness scripts/video_qa/record_feature.sh validate/ocr-analyze — tesseract OCR over sampled frames; self-validated (golden-good PASS / golden-bad FAIL) per §11.4.107(10)
Anti-bluff	every captured cast scanned for simulated/simulate/TODO implement/placeholder/for now/in production this would → **all clean**

Analyzer self-validation (the validator provably cannot bluff)

record_feature.sh selftest (§11.4.107(10)):

```
[golden-good] OCR-VERDICT: PASS (all expected patterns present, no bluff):
[golden-bad ] OCR-VERDICT: FAIL (missing expected pattern(s): HELIXCODE_WR
ANALYZER SELF-VALIDATION: PASS (golden-good PASS rc=0, golden-bad FAIL rc=
```

Every PASS below is read back by **this same** self-validated OCR analyzer.

Calibration note (§11.4.6 — patterns calibrated on the project's own frames)

tesseract renders this terminal font with a few stable substitutions: a space-prefixed 0→@ (so HelixCode 0.1.0→HelixCode @.1.0, Active Workers: 0→Active Workers: @), and [exit=0]→lexit=0]. Expected patterns were therefore chosen from tokens that survive OCR **and** uniquely prove the real feature (e.g. `go: go1.26.2`, `Total Memory: 0.00 GB`, `HELIXCODE_CMD_PROOF_123`, `42`). Emoji glyphs (📊 ✅ ⚠️) do not OCR and are not relied on. This is a calibration of the *patterns*, never of the *content* — the underlying real output is captured verbatim in the supplementary `.cast` files.

Durable evidence (committed) — rotation-proof anchors (§11.4.83, HXC-108 audit F2 fix)

The raw corpus (`/Volumes/T7/Downloads/Recordings/`, §11.4.128/.154-rotatable) is the secondary location. The three load-bearing BLUFF-clearing recordings + key frames are **copied into the committed tree** so a rotation cannot dangle these citations (per the android-doc best-practice the HXC-108 evidence audit flagged):

Committed artifact	sha256
`docs/qa/HXC-108_cli/helixcode-cli-version-20260622T210649Z.mp4` (h264 790×560 35f)	`7ca44a11e491ae809ab3420`
`docs/qa/HXC-108_cli/helixcode-cli-version-20260622T210649Z.evidence_frame.png`	`b014785fbb0badc92a2fd24`
`docs/qa/HXC-108_cli/helixcode-cli-generate-20260622T210841Z.mp4` (h264 790×560 68f)	`1d170e33f82373ed9849e60`
`docs/qa/HXC-108_cli/helixcode-cli-generate-20260622T210841Z.evidence_frame.png`	`07a132b1802ecd342c70f5d`
`docs/qa/HXC-108_cli/helixcode-cli-command_exec-20260622T211241Z.mp4` (h264 790×560 33f)	`68df1efdc86882e0b5ffc2d`
`docs/qa/HXC-108_cli/helixcode-cli-command_exec-20260622T211241Z.evidence_frame.png`	`3ece74ff62e13daa08a505b`

All three MP4s ffprobe-verified valid H.264 after copy. The remaining recordings (list-models, health, list-workers) live in the raw corpus only (non-BLUFF-class, secondary).

PER-FEATURE RESULTS

1. version — `cli -version` — PASS (deterministic)

- **Command:** `./bin/cli -version`
- **MP4:** `helixcode-cli-version-20260622T210649Z.mp4` (md5 `6e2317d99dd2f4a6f76a5b49cd753ae6`, H.264/yuv420p/35f)
- **OCR-validated real output excerpt** (read back from the recording):

```
HelixCode — AI development, end to end
HelixCode 0.1.0 (commit: dev, built: unknown, go: go1.26.2)
[exit=0]
```

- **Validator verdict:** `RECORDING-VERDICT: PASS` — `--expect "AI development, end to end|go: go1.26.2|exit=0"`. The real build string `go: go1.26.2` proves the genuine compiled banner.

2. list-models — `cli -list-models` — PASS (BLUFF-002 cleared)

- **Command:** `./bin/cli -list-models`
- **MP4:** `helixcode-cli-list_models-20260622T210824Z.mp4` (md5 `4751ac5ab1d1394cad55407a27e3a13a`, H.264/yuv420p/44f)
- **Real provider behaviour captured** (NOT a hardcoded list — real `providerManager.GetProviders()` + live `/models`):

```
DeepSeek provider initialized with 3 models
F12 provider: using "deepseek"
=== Available Models ===
DeepSeek catalog refreshed with 2 models (live /models)
ID: deepseek-v4-flash   Provider: deepseek   Context Size: 128000   St
ID: deepseek-v4-pro     Provider: deepseek   Context Size: 128000   St
ID: mistral-large       Provider: mistral    Score: SC:7.8    Context
ID: gemini-2.5-pro      Provider: gemini     Score: SC:8.7    Context
ID: mimo-v2.5-pro       Provider: xiaomi     Score: SC:8.1    Context
```

- **Validator verdicts (two complementary asserts, both from this one recording):**
    - Provider-init proof (frame extracted at recording time, harness `ocr-analyze`): `OCR-VERDICT: PASS` — `--expect "DeepSeek provider initialized|deepseek|Available Models"`.
    - Real-catalog proof (harness `validate`): `RECORDING-VERDICT: PASS` — `--expect "mistral-large|gemini-2.5-pro|Context Size: 128000|exit=0"`.
- **Note:** in a 24-row terminal the output scrolls; the 1 fps `validate` sampler caught the catalog tail in its sampled frames, and the init/`live /models` lines were verified on a higher-fps frame from the same MP4 via the identical self-validated analyzer. The real catalog (with per-model real scores/context-sizes) is the load-bearing BLUFF-002 proof.

3. generate — `cli -prompt … -model deepseek-v4-flash` — PASS (BLUFF-001 cleared)

- **Command:** `./bin/cli -prompt "What is 2+2? Answer with just the number." -model deepseek-v4-flash -max-tokens 50`
- **MP4:** `helixcode-cli-generate-20260622T210841Z.mp4` (md5 `b7a5664f296a2dac9cc8de5f50ee1578`, H.264/yuv420p/68f)
- **Real LLM transaction captured on one screen** (all OCR-readable):

```
=== Generating with deepseek-v4-flash ===
Prompt: What is 2+2? Answer with just the number.
4
DeepSeek catalog refreshed with 2 models (live /models)
tokens: in=17 out=18 total=35   context: 35/128000 (0%)   time: 1.36s
```

```
Generation completed
[exit=0]
```

- **Validator verdict:** `RECORDING-VERDICT: PASS` — `--expect "Answer with just the number|live /models|finish: stop|exit=0"`. The real model answer `4`, the live `/models` refresh, and the genuine token accounting (`in=17 out=18`, `time: 1.36s`, `finish: stop`) prove a real DeepSeek call — **not** a "simulated response".

4. health — `cli -health` — PASS

- **Command:** `./bin/cli -health`
- **MP4:** `helixcode-cli-health-20260622T211234Z.mp4` (md5 `b727ef1ddb4b99dd76b4933cd5cc4fb1`, H.264/yuv420p/37f)
- **OCR-validated real output:**

```
=== System Health Check ===
⚠️ Worker Pool: No healthy workers
⚠️ Notification System: No enabled channels
✅ System is operational
[exit=0]
```

- **Validator verdict:** `RECORDING-VERDICT: PASS` — `--expect "System Health Check|System is operational|exit=0"`. Honest real state (it reports the genuine absence of workers/notification channels, not a faked all-green).

5. list-workers — `cli -list-workers` — PASS

- **Command:** `./bin/cli -list-workers`
- **MP4:** `helixcode-cli-list_workers-20260622T211238Z.mp4` (md5 `3711b290051a5894bb399767a42a9b5d`, H.264/yuv420p/33f)
- **OCR-validated real output:**

```
=== Worker Statistics ===
Total Workers: 0    Active Workers: 0    Healthy Workers: 0
Total CPU: 0    Total Memory: 0.00 GB    Total GPU: 0
[exit=0]
```

- **Validator verdict:** `RECORDING-VERDICT: PASS` — `--expect "Worker Statistics|Total Workers: 0|Total Memory: 0.00 GB"`. Honest empty-pool state (no workers enrolled here).

6. command execution — `cli -command ...` — PASS (BLUFF-003 cleared)

- **Command:** `./bin/cli -command "echo HELIXCODE_CMD_PROOF_123 && expr 6 * 7"`
- **MP4:** `helixcode-cli-command_exec-20260622T211241Z.mp4` (md5 `c49bf4ea02dd28eeae6c988642d5b463`, H.264/yuv420p/33f)

- **OCR-validated real output:**

```
=== Executing Command ===
Command: echo HELIXCODE_CMD_PROOF_123 && expr 6 * 7
HELIXCODE_CMD_PROOF_123
42
✅ Command completed (exit code: 0)
[exit=0]
```

- **Validator verdict:** `RECORDING-VERDICT: PASS` — `--expect "Executing Command|HELIXCODE_CMD_PROOF_123|Command completed (exit code: 0)"`. **Strongest anti-bluff proof in the batch**: the shell genuinely *computed* `expr 6 * 7 = 42` (real `os/exec`, real exit code) — impossible for a print-and-sleep simulation.

SKIP / not recorded here (honest §11.4.3)

- **Streaming generate** (`-stream`): not separately recorded; non-streaming generate already proves the real LLM path. Recordable now in a follow-up if the conductor wants the streaming variant captured distinctly.
- `-qa-run` (QA session start): requires a reachable `qa-server` (default `http://localhost:8080`) — server-infra-gated; honest SKIP rather than a faked PASS. `-qa-list` recordable now if desired.
- **Server `/api/v1/llm/generate`, Desktop GUI, mobile**: out of this CLI slice (Desktop GUI is HXC-112-gated; server needs infra up).

Env gap (the reason for the asciinema route, §11.4.3)

The committed harness's native window-video path (`screencapture -v -V<n> -l<wid>` and the still-timelapse fallback `screencapture -o -l<wid>`) is **TCC-blocked** on this host:

```
screencapture: capture error The operation could not be completed
could not create image from window
```

The harness correctly refuses a whole-desktop fallback (§11.4.154) and SKIPs its live path. The asciinema text-capture route used here needs **no** Screen-Recording grant and produces the same MP4 + OCR-validation, so all CLI features were recorded TCC-free.

Sources verified

- `helix_code/bin/cli -help` (real flag inventory) — 2026-06-22.
- Direct runs of each feature before recording (confirmed genuinely working before any recording, §11.4.6) — 2026-06-22.
- `docs/qa/HXC-108_video_qa_matrix.md` (authoritative CLI×feature scope).